



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 10 по дисциплине «Защита информация»

Тема Создание и проверка электронной подписи документа

Студент Шахнович Дмитрий Сергеевич

Группа ИУ7-72Б

Преподаватель Руденкова Ю.С.

Москва, 2025

Содержание

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Определения	4
1.1.1 Простая электронная подпись	4
1.1.2 Неквалифицированная электронная подпись	4
1.1.3 Квалифицированная электронная подпись	4
1.2 Сравнение видов электронной подписи	5
1.3 Особенности усиленной неквалифицированной электронной подписи	5
1.4 Хэширование с SHA-256	5
1.5 Асимметричное шифрование RSA и PSS-подпись	6
1.6 Сертификаты X.509	6
1.7 Вывод	6
2 Технологическая часть	7
2.1 Средства реализации	7
2.2 Алгоритм формирования НЭП	7
2.3 Алгоритм проверки подписи	7
2.4 Код программы	7
2.5 Описание работы программы	13
2.6 Пример работы программы	14

ВВЕДЕНИЕ

Цель работы: Создание и проверка простой электронной подписи.

Задачи:

- 1) проанализировать виды электронной подписи;
- 2) разработать алгоритм подписывания документов неквалифицированной усиленной электронной подписью;
- 3) разработать алгоритм неквалифицированной усиленной простой электронной подписи;
- 4) обеспечить подпись произвольного текстового файла и его проверку.

1 Аналитическая часть

1.1 Определения

В соответствии с Федеральным законом № 63-ФЗ «Об электронной подписи» различают три вида электронной подписи: простую, неквалифицированную и квалифицированную.

1.1.1 Простая электронная подпись

Простая электронная подпись — электронная подпись, которая посредством использования кодов, паролей или иных средств подтверждает факт формирования электронной подписи определённым лицом (статья 5, пункт 2 Федерального закона № 63-ФЗ).

Данный вид подписи не требует применения средств криптографической защиты и обеспечивает только идентификацию подписанта, но не гарантирует целостность подписанного документа.

Примеры реализации простой электронной подписи:

- ввод логина и пароля в личном кабинете;
- подтверждение через SMS-код;
- отметка «согласен» на веб-форме;
- добавление в файл хэша пароля пользователя.

1.1.2 Неквалифицированная электронная подпись

Неквалифицированная электронная подпись — электронная подпись, которая:

- 1) получена в результате криптографического преобразования информации с использованием ключа электронной подписи;
- 2) позволяет определить лицо, подписавшее электронный документ;
- 3) позволяет обнаружить факт внесения изменений в электронный документ после момента его подписания;
- 4) создаётся с использованием средств электронной подписи.

(статья 5, пункт 3 Федерального закона № 63-ФЗ).

Таким образом, неквалифицированная подпись уже использует криптографические методы, обеспечивает контроль целостности и может применяться в большинстве внутренних информационных систем.

1.1.3 Квалифицированная электронная подпись

Квалифицированная электронная подпись — электронная подпись, которая соответствует всем признакам неквалифицированной электронной подписи и обладает следующими дополнительными признаками:

- 1) ключ проверки электронной подписи указан в квалифицированном сертификате;

2) для создания и проверки электронной подписи используются средства электронной подписи, имеющие подтверждение соответствия требованиям, установленным Федеральным законом № 63-ФЗ.

Квалифицированная подпись имеет полную юридическую силу и используется при взаимодействии с государственными органами и при оформлении юридически значимых документов.

1.2 Сравнение видов электронной подписи

Признак	ПЭП	НЭП	КЭП
Использует криптографию	Нет	Да	Да
Позволяет определить подписанта	Да (по паролю, коду)	Да	Да
Позволяет проверить целостность документа	Нет	Да	Да
Использует сертифицированные средства	Нет	Нет	Да
Юридическая сила	По соглашению сторон	Средняя	Полная

Таблица 1.1 — Сравнение видов электронной подписи согласно 63-ФЗ

1.3 Особенности усиленной неквалифицированной электронной подписи

В данной работе используется **усиленная неквалифицированная подпись**, которая:

- идентифицирует владельца через сертификат X.509 с публичным ключом и именем подписанта;
- обеспечивает целостность документа через хэширование содержимого файла и проверку подписи;
- защищена от подделки, так как подпись формируется приватным ключом RSA и проверяется публичным ключом.

1.4 Хэширование с SHA-256

Для проверки целостности файла используется алгоритм **SHA-256**:

- SHA-256 — криптографическая хэш-функция, выдающая 256-битное значение для любого входного сообщения;
- хэш-функция является односторонней и чувствительной к изменению данных: даже изменение одного бита файла приводит к совершенно другому хэшу;
- в программе хэш вычисляется от всего содержимого файла и подписывается приватным ключом RSA, что предотвращает подделку документа.

1.5 Асимметричное шифрование RSA и PSS-подпись

Для формирования подписи используется алгоритм **RSA** с модификацией **PSS (Probabilistic Signature Scheme)**:

- RSA — асимметричный алгоритм, использующий пару ключей: приватный для подписи и публичный для проверки подписи;
- RSA-PSS добавляет случайный «соль» к хэшируемому сообщению перед подписью, повышая стойкость подписи к криптоаналитическим атакам;
- в программе приватный ключ используется для создания подписи SHA-256 хэша файла, а публичный ключ из сертификата X.509 — для проверки подписи.

1.6 Сертификаты X.509

X.509 — стандарт цифровых сертификатов, содержащий публичный ключ и информацию о владельце:

- Common Name (имя подписанта);
- срок действия сертификата;
- назначение ключа и дополнительные поля.

В программе сертификат включается в блок подписи файла, что позволяет:

- определить владельца подписи;
- проверить корректность подписи;
- обеспечить связь подписи с конкретной личностью.

1.7 Вывод

Используемый набор алгоритмов (SHA-256 + RSA-PSS + X.509) позволяет создать **усиленную неквалифицированную электронную подпись**, которая:

- защищает от подделки;
- обеспечивает идентификацию подписанта;
- контролирует целостность документа.

Однако, такой сертификат всё ещё можно подменить, так как он не выдан доверенными центрами, которые подтверждали бы владением ресурса автора подписи, и так как нет цепочки доверия, то сертификат в файле можно легко подменить другим сертификатом, при этом подменив и файл.

2 Технологическая часть

2.1 Средства реализации

В качестве языка программирования выбран Go. Go обладает встроенными средствами работы с файлами, строками и криптографией (пакет `golang.org/x/crypto`), что делает его удобным для реализации НЭП.

2.2 Алгоритм формирования НЭП

Алгоритм формирования подписи выполняется следующим образом:

- 1) Пользователь выбирает приватный ключ (`private.pem`) и соответствующий сертификат X.509 (`cert.pem`).
- 2) Считывается содержимое файла, который необходимо подписать.
- 3) Вычисляется SHA-256 хэш всего содержимого файла.
- 4) Хэш подписывается приватным ключом RSA методом PSS (`rsa.SignPSS`).
- 5) Формируется блок подписи:
 - Base64-кодированная подпись,
 - сертификат X.509, содержащий публичный ключ и идентификационную информацию подписанта.
- 6) Блок подписи добавляется в конец файла, после чего файл сохраняется как новый, подписанный.

2.3 Алгоритм проверки подписи

Алгоритм проверки подписи:

- 1) Считывается подписанный файл и выделяется содержимое файла и блок подписи.
- 2) Из блока подписи извлекается сертификат X.509 и сама подпись.
- 3) Вычисляется SHA-256 хэш содержимого файла.
- 4) Проверка подписи выполняется с помощью публичного ключа из сертификата методом RSA-PSS (`rsa.VerifyPSS`).
- 5) Если проверка прошла успешно, выводится информация о подписанте (Common Name из сертификата) и подтверждается целостность файла.
- 6) В случае несоответствия подписи или повреждения файла выводится сообщение об ошибке.

2.4 Код программы

Листинг 2.1 — Код создания и проверки подписи

```
package main
```

```

import (
    "bufio"
    "bytes"
    "crypto"
    "crypto/rand"
    "crypto/rsa"
    "crypto/sha256"
    "crypto/x509"
    "crypto/x509/pkix"
    "encoding/base64"
    "encoding/pem"
    "fmt"
    "math/big"
    "os"
    "strings"
    "time"
)

const signatureMarker = "\n——BEGIN SIMPLE SIGNATURE——\n"
const signatureEnd = "——END SIMPLE SIGNATURE——"

// _____ genkeys _____

func genKeys(dir, commonName string) error {
    priv, err := rsa.GenerateKey(rand.Reader, 2048)
    if err != nil {
        return err
    }

    serial, err := rand.Int(rand.Reader, new(big.Int).Lsh(big.NewInt(1),
        128))
    if err != nil {
        return err
    }

    notBefore := time.Now().Add(-time.Minute)
    notAfter := time.Now().AddDate(1, 0, 0)

    template := x509.Certificate{
        SerialNumber: serial,
        Subject:      pkix.Name{CommonName: commonName},
    }

```



```

    NotBefore:    notBefore ,
    NotAfter:     notAfter ,
    KeyUsage:     x509.KeyUsageDigitalSignature | x509.
        KeyUsageKeyEncipherment ,
    ExtKeyUsage:  []x509.ExtKeyUsage{x509.ExtKeyUsageClientAuth , x509.
        ExtKeyUsageEmailProtection} ,
    IsCA:         false ,
}

derBytes , err := x509.CreateCertificate(rand.Reader , &template , &
    template , &priv.PublicKey , priv)
if err != nil {
    return err
}

privPem := pem.EncodeToMemory(&pem.Block{Type: "RSA PRIVATE KEY" , Bytes:
    x509.MarshalPKCS1PrivateKey(priv)})
if err := os.WriteFile(dir+"/private.pem" , privPem , 0600); err != nil {
    return err
}

certPem := pem.EncodeToMemory(&pem.Block{Type: "CERTIFICATE" , Bytes:
    derBytes})
if err := os.WriteFile(dir+"/cert.pem" , certPem , 0644); err != nil {
    return err
}

fmt.Println("Generated keys in:" , dir)
return nil
}

// ----- sign -----

func sign(privPath , certPath , inFile , outFile string) error {
    data , err := os.ReadFile(inFile)
    if err != nil {
        return err
    }

    shaSum := sha256.Sum256(data)

```

```

privPem, err := os.ReadFile(privPath)
if err != nil {
    return err
}
block, _ := pem.Decode(privPem)
if block == nil {
    return fmt.Errorf("invalid private key PEM")
}
privKey, err := x509.ParsePKCS1PrivateKey(block.Bytes)
if err != nil {
    return err
}

signature, err := rsa.SignPSS(rand.Reader, privKey, crypto.SHA256,
    shaSum[:], &rsa.PSSOptions{SaltLength: rsa.PSSSaltLengthEqualsHash})
if err != nil {
    return err
}
sigB64 := base64.StdEncoding.EncodeToString(signature)

certPem, err := os.ReadFile(certPath)
if err != nil {
    return err
}

var sb strings.Builder
sb.WriteString(signatureMarker)
sb.WriteString("SIGNATURE=")
sb.WriteString(sigB64)
sb.WriteString("\nCERT_BEGIN\n")
sb.Write(certPem)
if !strings.HasSuffix(string(certPem), "\n") {
    sb.WriteString("\n")
}
sb.WriteString("CERT_END\n")
sb.WriteString(signatureEnd)
sb.WriteString("\n")

out := append(data, []byte(sb.String())...)
return os.WriteFile(outFile, out, 0644)
}

```

```
// ----- verify -----

func verify(signedFile string) error {
    data, err := os.ReadFile(signedFile)
    if err != nil {
        return err
    }

    parts := bytes.SplitN(data, []byte(signatureMarker), 2)
    if len(parts) != 2 {
        return fmt.Errorf("no signature block found")
    }
    content := parts[0]
    sigPart := string(parts[1])

    var sigB64, certPEM string
    scanner := bufio.NewScanner(strings.NewReader(sigPart))
    inCert := false
    var certBuf strings.Builder
    for scanner.Scan() {
        line := scanner.Text()
        if line == signatureEnd {
            break
        }
        if line == "CERT_BEGIN" {
            inCert = true
            continue
        }
        if line == "CERT_END" {
            inCert = false
            continue
        }
        if inCert {
            certBuf.WriteString(line)
            certBuf.WriteString("\n")
            continue
        }
        if strings.HasPrefix(line, "SIGNATURE=") {
            sigB64 = strings.TrimPrefix(line, "SIGNATURE=")
        }
    }
}
```

```

}
certPEM = certBuf.String()
if sigB64 == "" || certPEM == "" {
    return fmt.Errorf("signature block missing required fields")
}

block, _ := pem.Decode([]byte(certPEM))
if block == nil || block.Type != "CERTIFICATE" {
    return fmt.Errorf("certificate PEM parse failed")
}
cert, err := x509.ParseCertificate(block.Bytes)
if err != nil {
    return err
}

sigBytes, err := base64.StdEncoding.DecodeString(sigB64)
if err != nil {
    return err
}

shaSum := sha256.Sum256(content)
pubKey, ok := cert.PublicKey.(*rsa.PublicKey)
if !ok {
    return fmt.Errorf("certificate public key is not RSA")
}
if err := rsa.VerifyPSS(pubKey, crypto.SHA256, shaSum[:], sigBytes, &rsa.PSSOptions{SaltLength: rsa.PSSSaltLengthEqualsHash}); err != nil {
    return fmt.Errorf("RSA-PSS verification failed: %w", err)
}

fmt.Printf("Signature valid.\nSigned by: %s\n", cert.Subject.CommonName)
return nil
}

// ----- main -----

func main() {
    if len(os.Args) < 2 {
        fmt.Println("Usage: genkeys/sign/verify ...")
        os.Exit(2)
    }
}

```

```

switch os.Args[1] {
case "genkeys":
    if len(os.Args) != 4 {
        fmt.Println("Usage: genkeys <dir> <commonName>")
        os.Exit(2)
    }
    if err := genKeys(os.Args[2], os.Args[3]); err != nil {
        fmt.Fprintln(os.Stderr, "genkeys error:", err)
        os.Exit(1)
    }
case "sign":
    if len(os.Args) != 6 {
        fmt.Println("Usage: sign <priv.pem> <cert.pem> <in> <out>")
        os.Exit(2)
    }
    if err := sign(os.Args[2], os.Args[3], os.Args[4], os.Args[5]); err !=
        nil {
        fmt.Fprintln(os.Stderr, "sign error:", err)
        os.Exit(1)
    }
case "verify":
    if len(os.Args) != 3 {
        fmt.Println("Usage: verify <signedfile>")
        os.Exit(2)
    }
    if err := verify(os.Args[2]); err != nil {
        fmt.Fprintln(os.Stderr, "verify error:", err)
        os.Exit(1)
    }
default:
    fmt.Println("Unknown command")
    os.Exit(2)
}
}

```

2.5 Описание работы программы

Программа состоит из трёх режимов:

- 1) **genkeys** — создаёт новую пару RSA-ключей и сертификат X.509 в указанной директории;

- 2) **sign** — создаёт новый файл, содержащий исходные данные и подпись;
- 3) **verify** — проверяет подпись файла с использованием публичного ключа из сертификата.

Пример запуска:

```
# Генерация ключей
./reinforced-sign genkeys ./keys "Ivan Ivanov"

# Подписание файла
./reinforced-sign sign ./keys/private.pem ./keys/cert.pem input.txt signed.txt

# Проверка подписи
./reinforced-sign verify signed.txt
```

При создании подписи в конец файла добавляется блок следующего вида:

```
---BEGIN SIMPLE SIGNATURE---
SIGNATURE=<Base64 подпись>
CERT_BEGIN
-----BEGIN CERTIFICATE-----
MIID... (сертификат в PEM)
-----END CERTIFICATE-----
CERT_END
---END SIMPLE SIGNATURE---
```

2.6 Пример работы программы

В качестве примера программы был взят текст из книги «Принципы юнит-тестирования» автора Владимир Хориков, длиной примерно 13000 символов.

На рисунке 2.1 представлен процесс подписи файла, проверки подписи и проверка подписи при изменении содержимого файла.

```
tail -n 7 test.txt | bat --paging=never

STDIN

1 Assert.Equal(5, result);
2 }
3 Метрика branch coverage показывает 100 %, и при этом тест проверяет все состав-
4 льящие результата метода. Такая составляющая здесь всего одна – возвращаемое значение. В то же время такой тест совершенно не является исчерпывающим: он не
5 учитывает ветвления, через которые может проходить метод .NET Framework int.
6 Parse. В то же время даже такой простой метод может содержать большое количество
7 ветвлений, как видно из рис. 1.6.

~/Projects/InfoSecurity main*
>
go run ./cmd/reinforced-sign/ sign ./config/sign/private.pem ./config/sign/cert.pem ./test.txt ./test.sig

~/Projects/InfoSecurity main*
>
tail -n 25 test.sig | bat --paging=never

STDIN

1 Parse. В то же время даже такой простой метод может содержать большое количество
2 ветвлений, как видно из рис. 1.6.
3 ---BEGIN SIMPLE SIGNATURE---
4 SIGNATURE=gLmf2+7RhZ4hpM0A1Ufwwx1X9v6/xIIV/ew9vstpq5FnYwfzgs0eb5nb0DZszU4usziESL3QkntPqCG/hqIF+hwSqZgnd0aErJUHJ+woWkG0vVM1774YA3KARteADETQzq51yVFh1mid44M
5 vRA1ItIKIgupeYDh4uNnBXzr70wreHQ==
6 CERT_BEGIN
7 -----BEGIN CERTIFICATE-----
8 MIIC2TCCAcGgAwIBAgIQSj62IJIIdfuK1ho1h/ZETeTANBgkqhkiG9w0BAQsFADAP
9 MQ0wCwYDVQQDEwRpbXB4XD1IMTEXNjIjYmZQwOVoXDTI2MTEXNjIjYmZUwOVow
10 DZENMASGA1UEAaMwIwTCCAS1wQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEB
11 ALZB0FY6f++1PcgJ41J24qDYINKjaA1POyzg3SgDwH70rr9g23w5P6c2EoAnM1
12 EZd0B1jzhEuIFuGIPNmm4xkh6QZD8dhPXr8n7TT8qgiJpG8I1xrGaxjUH4XX0
13 z15eLM0AzySoPPBjwCdaWFee1J60r8D0xRQcSPILY/vViKJJ+/wbbivfgqIH1It
14 Sr0Ux7CXJwaRqL0ok0soIDhuTLtIQkHg1tuntQA1Gi5s51yI6y3gH5PwH6ohot
15 kChwE52LqQdZ9MwLpw0b2PM3t5L1k0kthFyjq/xQC7ifent5j51COE6x6MwZP0fr
16 ha0cXWYA7Kc9Bq/bj/Xt+1MCwEAaMwMC2wDgIDVR8RPAQH/BAQDAgJhMB0GA1Ud
17 JQQMBQGCCGAQUFBwMKBgggrBgEFBQcDDBANBgkqhkiG9w0BAQsFAAOCAQEAR14e
18 LegXT12IndDnFU0491Gy/7PqQmR31twk+0GuBZtg3ztowof+4dkcKVC1abC71TS2
19 k1iFPKkbXcIUzrBu2ENLEW5MMh7vVtG3wSKVDJqR3RvyOfdVhHqNH4hyMhuhs5Vh
20 Fc+eBQ/q7GpBtmrkaXjsrLYYJ6wgpTgs0/MWn47eTBy6qsK7JU3wx0XfsH4zvdC
21 XYdtFcYLx1JbRvh7ViFmLK0weHOPgHEsmxiBMn6b0fWZ4qjdChM7kdTYNLgKbc5
22 0ZDITyJEbtfcFtQLSxofkSzinqo4wMAT8u+SaFB+ac8+HkuuVRZaoQLob9v5V3h
23 sRAOW84ZCheEggsbzw==
24 -----END CERTIFICATE-----
25 CERT_END
26 ---END SIMPLE SIGNATURE---

~/Projects/InfoSecurity main*
>
go run ./cmd/reinforced-sign/ verify ./test.sig
Signature valid
Signed by: impi

~/Projects/InfoSecurity main*
>
sed 's/b/a/g' test.sig > test.sig

~/Projects/InfoSecurity main*
>
go run ./cmd/reinforced-sign/ verify ./test.sig
verify error: no signature block found
exit status 1
```

Рисунок 2.1 — Подпись и проверка текстового файла

В качестве примера шифрования архива был взят архив с кодом текущей и предыдущей лабораторных работ. На рисунке 2.2 представлен процесс подписи и проверки подписи архивного файла.

```

~
• whoami
  impervguin

~/Projects/InfoSecurity main*
>
• go run ./cmd/reinforced-sign/ verify ./code.zip
  verify error: no signature block found
  exit status 1

~/Projects/InfoSecurity main*
>
• go run ./cmd/reinforced-sign/ sign ./config/sign/private.pem ./config/sign/cert.pem ./code.zip ./code.

~/Projects/InfoSecurity main*
>
• go run ./cmd/reinforced-sign/ verify ./code.sig.zip
  Signature valid.
  Signed by: impi

```

Рисунок 2.2 — Подпись и проверка архивного файла

ЗАКЛЮЧЕНИЕ

В результате лабораторной работы была разработана программа для неквалифицированной электронной подписи.

Были выполнены следующие задачи:

- 1) проанализированы виды электронной подписи;
- 2) разработаны алгоритмы подписания и проверки подписи;
- 3) реализован алгоритм формирования неквалифицированной электронной подписи на языке Go;
- 4) реализована проверка подписи.

Все поставленные цели и задачи были выполнены.